

Pipeline and Agents

This document follows one selected C program through the system. The most important boundary is that visible evidence can guide repairs, while Judge evidence cannot.

Before Comparing Prompts

OxCidation first prepares one visible-test suite for the C program. This happens once, before translating the program with different prompts.

```
C source
-> LLM proposes input cases
-> each candidate is executed twice against C
-> duplicate, failed, timed-out, or unstable candidates are rejected
-> Validator reviews every candidate independently
-> approved cases are preserved; rejected cases request replacements once
-> all approved cases are frozen and shared by every prompt
```

The test generator receives only the C source. It does not receive the problem description, public examples, or Judge cases. It also does not generate the expected answers: those come from running the accepted C program. The suite review receives the C source, every candidate input, and the deterministic execution summary, but no Rust code, problem description, expected output, or Judge data.

Invalid cases and deterministic rejections request only the same number of replacement candidates, at most once. Previously approved cases are never regenerated or reviewed again. Invalid or inconclusive replacement cases are discarded; if at least one approved case remains, that partial suite is frozen. Visible testing is disabled only when no approved case remains. A terminal decision is reused on resume. This reduces malformed inputs, but cannot establish constraints that are absent from the C source.

A failed LLM request or malformed structured response is not a semantic `inconclusive` decision. It is recorded as `review_failed`, is not reusable, and stops the benchmark before any translation prompt runs for that program. A later `--resume` invocation retries the preparation step.

Processing One Program and Prompt

The flow has four phases.

Phase 1: Initial translation

1. The Translator receives the C source and one translation prompt.
2. The model returns Rust code.
3. Rust is compiled.
4. The initial code and compilation result are preserved.
5. The Evaluator records an initial Judge observation when Judge data is configured.

If Rust does not compile, the Validator may request a repair using compiler output. This repeats only until the configured repair limit.

Phase 2: Visible comparison

Once Rust compiles, the Tester executes C and Rust on the shared visible suite. The resulting report describes how many cases passed and which cases differ.

- If the visible cases pass, the code proceeds to final evaluation.
- If the test infrastructure is unavailable, the pipeline records that fact and proceeds without treating it as a Rust correctness failure.
- If C passes and Rust fails, the Validator performs semantic analysis.

Phase 3: Semantic repair

For a differential failure, the Validator receives:

- the original C source;
- the current Rust code;
- a compact test report with limited failure examples.

The Validator asks the LLM to classify the evidence:

Classification	Meaning	Next step
translation_discrepancy	Rust probably changed the behavior of C.	Send concise diagnosis and guidance to the Translator.
invalid_visible_test	The generated input probably does not represent a valid program input.	Do not modify Rust; continue to external evaluation.
inconclusive	The available evidence cannot support either conclusion.	Do not modify Rust; continue to external evaluation.

Approved repairs return to compilation and visible testing. The loop stops when the code passes, the repair limit is reached, or the evidence should not cause repair.

Phase 4: External evaluation

The Evaluator runs the final Rust code against the configured Judge suite. It first checks the original C program on the same cases. Rust is evaluated only when the C baseline is accepted.

The result may contain AC, WA, TLE, RE, MLE, or infrastructure failures, along with case counts and the first failure. The pipeline records this result and ends. Reaching the repair limit still produces a final Judge observation. The Evaluator never sends Judge failures to the Translator or opens another repair cycle.

If no repair changed the initial Rust code, the final result reuses the initial Judge observation instead of executing identical code twice.

Agent Responsibilities

The agents are roles in one coordinated process. They do not operate as independent conversational assistants.

Translator

Produces and changes Rust code.

Receives	Produces
C source and translation prompt	Initial Rust code and raw model response
Current Rust and compiler error	Compilation repair

Current Rust and approved Validator guidance	Semantic repair
--	-----------------

Tester

Produces visible, language-agnostic behavioral evidence.

Receives	Produces
C source	Candidate visible inputs
Candidate inputs	Stored .in/.out suite using C as the output oracle
C, Rust, and the stored suite	Differential test report

Validator

Interprets evidence and chooses the next pipeline action.

Receives	Produces
Translation or compilation status	Continue, repair, or stop decision
C source, candidate inputs, and deterministic preparation report	Per-case approval, invalid-case diagnosis, or inconclusive decision
Differential visible failure	Semantic diagnosis, test assessment, and optional repair guidance
Judge result	Terminal classification for reporting only

The routing rules are deterministic. The two semantic judgments made by the configured model are the pre-translation suite review and the analysis of a differential visible failure.

Evaluator

Produces external observational evidence.

Receives	Produces
Original C, initial or final Rust, problem limits, and Judge cases	C/Rust comparison, verdict counts, and first failure

The Evaluator never produces repair guidance.

Where the LLM Participates

The configured model is called for:

1. visible-input generation;
2. pre-translation visible-suite review;
3. initial C-to-Rust translation;
4. semantic analysis of a differential visible failure;
5. an approved Rust repair.

Compilation, program execution, output comparison, verdict aggregation, flow routing, and report generation are implemented as deterministic operations.

Following a Repair in the Logs

For one program-prompt result, `agent_interactions.json` records events in chronological order. Sequence numbers and causal references connect the visible failure, Validator analysis, decision, repair, recompilation, and new test report. The file is the best source for understanding the complete feedback loop; final snapshot reports may show only the last attempt.